



# Revizor: Testing Black-Box CPUs against Speculation Contracts

Oleksii Oleksenko\*  
Christof Fetzer  
TU Dresden  
Dresden, Germany

Boris Köpf  
Microsoft Research  
Cambridge, UK

Mark Silberstein  
Technion  
Haifa, Israel

## ABSTRACT

Speculative vulnerabilities such as Spectre and Meltdown expose speculative execution state that can be exploited to leak information across security domains via side-channels. Such vulnerabilities often stay undetected for a long time as we lack the tools for systematic testing of CPUs to find them.

In this paper, we propose an approach to *automatically* detect microarchitectural information leakage in commercial black-box CPUs. We build on speculation contracts, which we employ to specify the permitted side effects of program execution on the CPU's microarchitectural state. We propose a Model-based Relational Testing (MRT) technique to empirically assess the CPU compliance with these specifications.

We implement MRT in a testing framework called Revizor, and showcase its effectiveness on real Intel x86 CPUs. Revizor automatically detects violations of a rich set of contracts, or indicates their absence. A highlight of our findings is that Revizor managed to automatically surface Spectre, MDS, and LVI, as well as several previously unknown variants.

## CCS CONCEPTS

• Security and privacy → Side-channel analysis and counter-measures.

## KEYWORDS

microarchitecture, contracts, spectre, mds, testing, information flow

### ACM Reference Format:

Oleksii Oleksenko, Christof Fetzer, Boris Köpf, and Mark Silberstein. 2022. Revizor: Testing Black-Box CPUs against Speculation Contracts. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)*, February 28 – March 4, 2022, Lausanne, Switzerland. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3503222.3507729>

\*Work partially done at Microsoft Research Cambridge.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).  
ASPLOS '22, February 28 – March 4, 2022, Lausanne, Switzerland  
© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.  
ACM ISBN 978-1-4503-9205-1/22/02...\$15.00  
<https://doi.org/10.1145/3503222.3507729>

## 1 INTRODUCTION

The instruction set architecture (ISA) specifies the functional behavior of a CPU but abstracts from its implementation details (microarchitecture). This abstraction enables rapid development of hardware optimizations without requiring changes to the software stack; unfortunately, it also obscures the security impact of these optimizations. Over the last decade researchers discovered numerous microarchitectural zero days, including Spectre-style attacks that use microarchitectural state to exfiltrate secret information obtained during transient execution [23, 26]. The problem is expected to get worse as Moore's law subsides and CPU architects are compelled to apply ever more aggressive optimizations [37].

Speculation contracts (short: contracts) [18] have been proposed as a way out of this situation by providing a *specification* of the microarchitectural side effects. Contracts declare which ISA operations an attacker can observe through a side channel, and which operations can speculatively change the control/data flow. For example, a contract may state: an attacker can observe addresses of memory stores and loads, and the CPU may mispredict the targets of conditional jumps. If a CPU implementation permits the attacker to observe *more* than that (e.g., addresses of loads after mispredicted *indirect* jumps), the CPU violates the contract, indicating an unspecified leak in the microarchitecture.

For software developers, contracts are a foundation for microarchitecturally secure programming: they spell out the assumptions that are required for checking that mitigations are effective and code is free of leaks. For example, a recent survey [9] classifies existing tools for detecting speculative vulnerabilities in the language of contracts. For hardware developers, contracts can provide a target specification that describes the permitted microarchitectural effects of the CPU's operations, without putting further constraints on the hardware implementation. Thus, contracts hold the promise to achieve for speculative vulnerabilities what consistency models have provided for memory consistency [3].

Despite the contracts' potential, so far they have only been used for establishing security guarantees of small white-box models of CPUs with toy ISAs [18]. In the context of real-world CPUs, several existing tools (e.g., Medusa [28], SpeechMiner [51], and CheckMate [41]) target automated detection of known types of speculative leaks, but not contract violations in general. Thus, it has been an open challenge to test contract compliance of real-world CPUs, with complex ISAs and absent (or intractable) models of the microarchitecture.

**Approach.** In this paper, we propose a method and a tool for testing real-world CPUs against speculation contracts. Our method, called *Model-based Relational Testing* (MRT), is a randomized search for "evidence" of contract violations, i.e., for counterexamples to contract compliance.

Such a counterexample is a specific instance where the CPU leaks more information than the contract permits. In particular, a counterexample is an instruction sequence together with a pair of inputs that produce the *same* observations according to the contract (*contract trace*), but *different* microarchitectural side-effects on the CPU (*hardware trace*).

MRT searches for counterexamples by creating samples—random instruction sequences (*test cases*) together with random inputs—and checking if any of them constitutes a counterexample. A key observation is that this check *does not* require an explicit model of the microarchitecture. This is because one only needs to compare traces of the same kind, that is, contract traces to contract traces, and hardware traces to hardware traces. This enables side-stepping the need for establishing a connection between them via a model of the microarchitecture (as done in [18]) and enables testing commercial black-box CPUs. However, the search for counterexamples on real-world CPUs poses a new set of challenges:

The first challenge is to cope with the *intractable search space*: ISAs typically include hundreds of instructions, dozens of registers, and permit large memory spaces. This creates an intractable number of possibilities for both test cases and for inputs to them. Moreover, there are no means to measure coverage for black-box CPUs, which precludes a guided search. We solve this problem by using an incremental generation process that aims to create ample opportunities for speculation: (1) We perform testing in rounds, where we start by generating short instruction sequences with few basic blocks, a small subset of registers, and where we confine all memory accesses to a narrow memory range. (2) After each round without counterexample, we invoke a diversity analysis that counts the number of tested instruction patterns that we expect to induce speculative leaks. This analysis triggers reconfiguration of the test generator to gradually expand the search space in subsequent testing rounds.

The second challenge is to obtain *deterministic hardware traces* from modern high-performance CPUs with complex and unpredictable microarchitectures. For this we (1) create a low-noise measurement environment where we execute test cases in complete isolation and perform a side-channel attack (e.g., Prime+Probe on the L1D cache) to detect leakage into the microarchitecture, and (2) we control the microarchitectural context using a technique we call *priming*: Priming collects traces for a large number of pseudorandom inputs to the same test case *in sequence*. In this way, execution with one input effectively sets the microarchitectural context for the next input. This enables collection of hardware traces with predictors primed in a diverse but deterministic fashion, which is key to obtaining comprehensive and stable hardware traces.

The third challenge is to *generate contract traces for complex ISAs* such as x86. To tackle this challenge we implement executable contracts by instrumenting an existing ISA emulator with a check-pointing mechanism similar to [32], which enables us to explore correct and mispredicted execution paths, and to record the contract-prescribed observations during program execution.

**Tool & Evaluation.** We implement MRF as a testing framework Revizor<sup>1</sup>. The current implementation supports only Intel x86, which

we chose as a worst-case target for our method: a superscalar CPU with several unpatched microarchitectural vulnerabilities, no detailed descriptions of speculation mechanisms, and no direct control over the microarchitectural state.

We evaluated Revizor on two different microarchitectures, Skylake and Coffee Lake, and with different microcode patches. We test these targets against a sequence of increasingly permissive contracts. This gradually filters out common violations, and narrows down on more subtle violations. The key highlights of our evaluation are:

- (1) When testing a patched Skylake against a restrictive contract that states that speculation exposes *no information*, Revizor detects a violation within a few minutes. Inspection shows the violation stems from the leakage during branch prediction, i.e. a representative of Spectre V1.
- (2) When testing Skylake with *V4 patch disabled* against a contract that permits leakage during branch prediction (and is hence *not* violated by V1), Revizor detects a violation due to address prediction, i.e., a representative of Spectre V4.
- (3) When further weakening the contract to permit leaks during both types of speculation, Revizor still detects a violation. This violation is a novel (minor) variant of Spectre where the timing of variable-latency instructions (which is *not* permitted to leak according to the contract) leaks into L1D through a race condition induced by speculation.
- (4) When making microcode assists possible during collection of the hardware traces, Revizor surfaces MDS [7, 44] on the same CPU and LVI-Null [43] on a CPU patched against MDS.
- (5) When used to validate an assumption that stores do not modify the cache state until they retire, made in recent defence proposals [46, 53], Revizor discovered that this assumption does not hold in Coffee Lake.
- (6) In terms of speed, Revizor processes over 200 test cases per hour for complex contracts, and with several hundreds of inputs per test case, which enables discovery of Spectre V1, V4, MDS, and LVI-Null in under two hours, on average.

**Summary.** In summary, starting from simple contracts, Revizor could automatically generate gadgets that represent all three of the known types of speculative leakage: speculation of control flow, address prediction, and speculation on hardware exceptions. Notably (and perhaps surprisingly), Revizor finds them within only a few hours of testing on an ordinary desktop PC, despite the enormous size of the search space. The reason is that counterexample search is *not* akin to finding a needle in a haystack. Instead, microarchitectural leaks manifest in many programs, and it is sufficient to find only one of them. This result demonstrates the practicality of testing complex real-world CPUs against speculation contracts.

The source code is publicly available under:

<https://github.com/hw-sw-contracts/revizor>

## 2 BACKGROUND: CONTRACTS

### 2.1 Hardware Traces and Side-channel Leakage

We consider an abstract side-channel attack model whereby an adversary can use side-channels [33, 42, 52] to extract secret information about a victim program *Prog* execution. Specifically, we

<sup>1</sup>Revizor is a name of a classical play by Nikolai Gogol about a government inspector arriving into a corrupt town for an incognito investigation.

**Table 1: Summary of MEM-COND. Note that the execution clause describes the speculative behavior of a conditional jump, as the jump takes place (IP is updated) if the condition is false, the opposite of the non-speculative execution.**

|            | Observation Clause | Execution Clause                                            |
|------------|--------------------|-------------------------------------------------------------|
| Load       | expose: ADDRESS    | None                                                        |
| Store      | expose: ADDRESS    | None                                                        |
| Cond. Jump | None               | speculate:<br>if (INVERTED_CONDITION){<br>IP = IP + TARGET} |
| Other      | None               | None                                                        |

focus on microarchitectural side-channels, such as cache timing. We define a **hardware trace** as a sequence of all the observations made through the side-channel after each instruction during a program execution.

We represent the hardware trace as the output of a function *Attack*

$$HTrace = Attack(Prog, Data, Ctx)$$

that takes three input parameters: (1) the victim program *Prog*; (2) the input *Data* processed by the victim’s program (i.e., the architectural state including registers and main memory); (3) the microarchitectural context *Ctx* in which it executes.

The information exposed by a hardware trace depends on the assumed side-channel and threat model.

*Example:* If the threat model includes attacks on a data cache, *HTrace* is composed of the cache set indexes used by *Prog*’s loads and stores. If it includes attacks on an instruction cache, *HTrace* contains the addresses of executed instructions.

A program **leaks** information via side-channels when its hardware traces depend on the inputs (*Data*): We assume the attacker knows *Prog* and can manipulate *Ctx*, hence any difference between the hardware traces implies difference in *Data*, which effectively exposes information to the attacker.

Intuitively, hardware traces encompass the microarchitectural leaks during the program execution on a given CPU, including speculative execution. For example, the trace will record a sensitive memory access during a branch misprediction, such as the leak exploited in Spectre [23].

## 2.2 Legitimate Exposure as a Contract

We now show how speculation contracts can be used to specify the information legitimately exposed by each instruction.

A **speculation contract** [18] specifies the information that can be exposed by a CPU during a program execution under a given threat model. For each instruction in the CPU ISA (or a subset thereof), a contract describes the information exposed by the instruction’s (**observation clause**) and the externally-observable speculation that the instruction may trigger (**execution clause**). When a contract covers a subset of ISA, the leakage of unspecified instructions is undefined.

*Example:* Consider a contract called *MEM-COND* (summarized in Table 1). Through the observation clauses of loads and stores, the contract prescribes that addresses of all memory access may be

```

1 z = array1[x] # base of array1 is 0x100
2 if (y < 10)
3   z = array2[y] # base of array2 is 0x200

```

**Figure 1: Example of Spectre V1**

exposed (hence *MEM*). The execution clause of conditional branches describes their misprediction, thus the contract prescribes that branch targets may be mispredicted (hence *COND*). This way, the contract models a data cache side channel on a CPU with branch prediction.

A **contract trace** *CTrace* contains the sequence of *all* the observations the contract allows to be exposed after each instruction during a program execution, including the instructions executed speculatively. Conversely, the information that is *not* exposed via *CTrace* is supposed to be kept secret.

We represent a contract as a function *Contract* that maps the program *Prog* and its input *Data* to a contract trace *CTrace*:

$$CTrace = Contract(Prog, Data)$$

*Example:* Consider the program in Figure 1, executed with an input *data*={*x*=10,*y*=20}. The *MEM-COND* contract trace is *ctrace*=[0x110, 0x220], representing that the load at line 1 exposes the accessed address during normal execution, and the load at line 3 exposes its address during speculative execution triggered by the branch at line 2.

A CPU complies [18] with a contract when its hardware traces (collected on the actual CPU) leak at most as much information as the contract traces. Formally, we require that whenever any two executions of *any* program have the same contract trace (implying the difference between inputs is not exposed), the respective hardware traces should also match.

**Definition 1.** A CPU **complies** with a *Contract* if, for all programs *Prog*, all input pairs (*Data*, *Data'*), and all initial microarchitectural states *Ctx*:

$$\begin{aligned}
&Contract(Prog, Data) = Contract(Prog, Data') \\
\Rightarrow &Attack(Prog, Data, Ctx) = Attack(Prog, Data', Ctx)
\end{aligned}$$

This approach is called *relational reasoning*, and is natural for expressing information flow properties [10]. In the corresponding terminology [39], Def 1 requires that any program that is non-interferent with respect to a contract must also be non-interferent on the CPU.

Conversely, a CPU **violates** a contract if there exists a program *Prog*, a microarchitectural state *Ctx*, and two inputs *Data*, *Data'* that agree on their contract traces but *disagree* on the hardware traces. We call the tuple (*Prog*, *Ctx*, *Data*, *Data'*) a contract **counterexample**. The counterexample witnesses that an adversary can learn more information from hardware traces than what the contract specifies. A counterexample indicates a potential microarchitectural vulnerability that was not accounted for by the contract.

*Example:* Consider a contract, called *MEM-SEQ*, which allows exposure of memory accesses (similarly to *MEM-COND*), but limits it to only non-speculative accesses. A CPU that leaks on speculatively

executed branches will violate *MEM-SEQ*. Its counterexample is the program in Figure 1 together with inputs  $\text{data1}=\{x=10, y=20\}$  and  $\text{data2}=\{x=10, y=30\}$  and a context that triggers a misprediction: The contract trace for both inputs is  $\text{ctrace}=[0x110]$ . However, when the CPU mispredicts the branch (line 2) and speculatively accesses memory (line 3), the hardware traces will diverge ( $\text{htrace1}=[0x110, 0x220]$  and  $\text{htrace2}=[0x110, 0x230]$ ). Yet, this is not a counterexample to *MEM-COND*, because its contract traces already expose the memory accesses on both paths of a branch.

### 2.3 Concrete Contracts of Speculation

A contract is constructed from a combination of an observation and execution clauses. We first describe individual clauses, and then show how they form concrete contracts.

#### Observation clauses:

- *MEM (Memory Address)*: exposes the addresses of data loads and stores. Represents a data cache timing side-channel attack.
- *CT (Constant-Time)*: extends MEM by additionally exposing Program Counter. Represents both data and instruction cache attacks. Based on a typical threat model for constant-time programming (hence the name), except it does not expose the execution time of variable-latency operations.
- *ARCH (Architectural Observer)*: extends CT by additionally exposing the *values* loaded from memory. Represents a same-address-space attack, such as assumed in the Speculative Taint Tracking paper [53].

#### Execution clauses:

- *SEQ*: observations are only collected during *sequential execution* (in-order, nonspeculative). This is a model of a processor that allows speculation but constrains the information leaked during the speculation when combined with the appropriate observation clause.
- *COND*: observations are also collected after *conditional jump misprediction*. That is, they are collected from both correct and mispredicted paths. The length of the mispredicted path is limited by a predefined speculation window.
- *BPAS*: observations are collected after *store bypass*: all stores are speculatively skipped. The mis-speculated execution rolls back after the speculation window as in *COND*.
- *COND-BPAS*: Combination of *COND* and *BPAS*.

**Full contracts.** We illustrate how the clauses form a contract with examples:

*Example: CT-COND* exposes addresses of all memory accesses and of all control-flow transitions, including those on mispredicted paths of conditional branches. *CT-COND* models a CPU vulnerable to Spectre V1 attacks.

*Example: ARCH-SEQ* exposes addresses and values of non-speculative loads and stores. There is a subtle difference from *MEM-SEQ*. While *MEM-SEQ* disallows speculative leakage of any values, *ARCH-SEQ* disallows leakage of only speculatively loaded values. This is equivalent to transient noninterference[53].

## 3 CHALLENGES OF TESTING CONTRACT COMPLIANCE

In this work, we leverage contracts to check compliance of complex commercial CPUs under realistic threat models. Assuming that a contract properly exposes the expected information leakage in a CPU, finding a counterexample would signify an unexpected, hence potentially exploitable, leakage.

While the original paper [18] proved compliance on an *abstract* CPU with toy assembly, testing compliance of a *real hardware* CPU with complex ISA poses significant challenges.

### 3.1 How to Find a Counterexample?

The search space for counterexamples is all possible programs, inputs, and all microarchitectural contexts. Such an immense search space cannot be explored exhaustively, thus requiring a targeted search.

**CH1: Binary Generation.** While a contract prescribes which instructions are permitted to speculate and expose information, we search for *unexpected* speculation and leakage, thus we need to collect traces that encompass all the instructions. Furthermore, a particular *sequence of instructions* is usually required to produce an observable leakage, thus we need to test different instruction sequences. Moreover, to trigger an *incorrect* speculation (e.g., a branch misprediction), we need to prime the microarchitectural state in diverse ways. All of it calls for a search strategy that tests diverse instruction sequences with diverse inputs, but with a priority to those that are likely to leak or to produce speculation.

**CH2: Input Generation.** For an input to be useful in forming a counterexample, we need another input that produces the same contract trace. Such inputs are called *effective inputs*. The *ineffective* inputs which produce a unique contract trace constitute a wasted effort as they cannot, by definition, reveal contract violation. This challenge calls for a more structured input generation approach rather than a simple random one, as the probability that multiple random inputs will produce the same contract trace is low.

### 3.2 How to Get Stable Hardware Traces on a Real CPU?

**CH3: Collection of Hardware Traces.** CPUs have no direct interface to record information leaked in hardware traces, such as addresses accessed in a speculative path. Thus, we have to perform indirect sampling-based measurements, which are inevitably imprecise and incomplete.

**CH4: Uncontrolled Microarchitectural State.** Black-box CPUs normally have no direct way to set the microarchitectural context for test execution as required by Def 1. For example, branch predictors are not accessible architecturally, and some are not even disclosed. Moreover, speculation depends on multiple, often unknown factors, such as fine-grained power saving [29, 38], or contention on shared resources. Thus, speculation can happen nondeterministically, and cause divergent traces without a real information leak (false positive). On the other hand, if the speculation is never triggered during the measurement, speculative leaks cannot be observed, leading to false compliance (false negative).

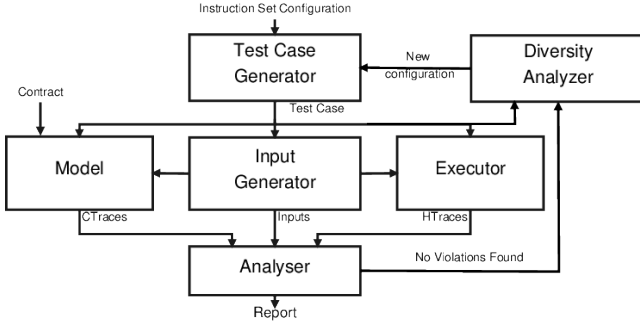


Figure 2: Main flow of Model-based Relational Testing.

**CH5: Noisy Measurements.** The measurements are influenced by neighbour processes on the system, by hardware mechanisms (e.g., prefetching), and by inherent imprecision of the measurement tools (e.g., timing measurements). This challenge differs from CH4 as it affects the measurement precision rather than the program execution. The noise may result in divergence between the otherwise equivalent traces, leading to a false positive.

### 3.3 How to Produce Contract Traces?

**CH6: Collection of Contract Traces.** All contracts in [18] are defined for a toy assembly; it is unclear how to collect traces for a contract describing a complex ISA. To allow realistic compliance check, we need work with real binaries generated via standard compiler tool chain. Hence, we need a method to automatically collect contract-prescribed observations for a given program executed with a given input.

## 4 MODEL-BASED RELATIONAL TESTING

We present Model-based Relational Testing (MRT), our approach to identifying contract violations in black-box CPUs. Here we provide a high-level description, with the technical details to follow (§5). Figure 2 shows the main steps.

**Test case and input generation.** We sample the search space of programs, inputs and microarchitectural states to find counterexamples. The generated instruction sequences (**test cases**) are comprised of the ISA subset described by the contract. The test cases and respective inputs to them are generated to achieve high diversity and to increase speculation or leakage potential (§5.1 and §5.2).

**Collecting contract traces.** We implement an executable Model of the contract to allow automatic collection of contract traces for standard binaries. For this, we modify a functional CPU emulator to implement speculative control flow based on a contract’s execution clause, and to record traces based on its observation clause (§5.4).

**Collecting hardware traces.** We collect hardware traces by executing the test case on the CPU under test and measuring the observable microarchitectural state changes during the execution according to the threat model. The executor employs several methods to achieve consistent and repeatable measurements (§5.3).

**Relational Analysis.** We analyze the contract and hardware traces to identify violations of Def 1. This requires *relational* reasoning:

- (1) We partition inputs into groups, which we call **input classes**. All inputs within a class have the same contract trace. Thus, input classes correspond to the equivalence classes of equality on contract traces. Classes with a single (ineffective) input are discarded.
- (2) For each class, we check if all inputs within a class have the same hardware trace.

If the check fails on any of the classes, we found a counterexample that witnesses contract violation (§5.5).

**Diversity-guided generation.** The testing process is performed in rounds, where earlier rounds exercise smaller search space (i.e., shorter instruction sequences, fewer basic blocks) to speed up testing. After each round that did not yield a counterexample, we invoke a test case diversity analysis which may trigger reconfiguration of the test generator to produce richer test cases, gradually expanding the search space (§5.6).

## 5 DESIGN AND IMPLEMENTATION

We build a tool Revizor that implements MRT for practical end-to-end testing of x86 CPUs against speculation contracts. We describe the individual components of Revizor and how they address the challenges outlined in §3.

### 5.1 Test Case Generator

The task of the test case generator is to sample the search space of all possible programs. As described in CH1, the sampling should be diverse, so that we have a chance to observe an *unexpected* leakage or speculation. Fully random generation, however, might lead to generating incorrect programs, e.g., with invalid control flow or memory accesses, leading to unhandled exceptions during their execution. This is why we rely on a randomized generation algorithm which imposes a certain structure on the generated instruction sequence and its memory accesses. It works as follows:

- (1) Generate a random Directed Acyclic Graph (DAG) of basic blocks;
- (2) Add jump instructions (terminators) at the end of basic block to ensure the control flow matches the DAG.
- (3) Add random instructions from the tested ISA subset;
- (4) Instrument instructions to avoid faults:
  - (a) mask memory addresses to confine them within a dedicated memory region, which we call *sandbox*;
  - (b) modify division operands to avoid division by zero;
- (5) Compile the test case into a binary.

The total number of instructions, functions, and basic blocks per test, as well as the tested instruction (sub)set are specified by the user. We borrow the ISA description from nanoBench [2].

**Example:** Figure 3 shows a test case example, produced in multiple steps: ① The generator created a DAG with three nodes. ② Connected the nodes by placing either conditional or direct jumps (lines 4–5, 10). ③ Added random instructions until a specified size was reached (lines 1, 3, 7–9, 13, 14). ④ Masked the memory accesses and aligned to the sandbox base in R14 (lines 2, 6, 12).

We use DAG as a basis for the generation process to confine the control flow and avoid infinite loops. The limitation of this approach is that we do not test loops, which may prevent Revizor

```

1 OR RAX, 468722461
2 AND RAX, 0b111111000000
3 LOCK SUB byte ptr [R14 + RAX], 35
4 JNS .bb1
5 JMP .bb2
6 .bb1: AND RCX, 0b111111000000
7 REX SUB byte ptr [R14 + RCX], AL
8 CMOVNBE EBX, EBX
9 OR DX, 30415
10 JMP .bb2
11 .bb2: AND RBX, 1276527841
12 AND RDX, 0b111111000000
13 CMOVBE RCX, qword ptr [R14 + RDX]
14 CMP BX, AX

```

**Figure 3: Randomly generated test case**

from detecting loop-based contract violations. However, it is only a technical limitation and, in the future, it could be solved by analyzing the control flow of test cases and enforcing loop termination at generation time.

**Improving input effectiveness.** Using many hardware registers and larger sandbox results in low input effectiveness (CH2), as it increases the likelihood of unique contract traces that cannot be used for relational testing. To improve input effectiveness, the generator generates programs with only four registers, confines the memory sandbox to one or two 4K memory pages, and aligns memory accesses to a cache line (64B). To test different alignments, the accesses are further offset by a random value between 0 and 64 (the same within a test case but different across test cases).

## 5.2 Input Generator

An input is a set of values to initialize the architectural state, which includes registers (including FLAGS) and the memory sandbox. Revizor creates random inputs with a 32-bit PRNG.

The initial number of inputs per test case is configured up-front, and it increases every time the diversity analyser triggers a reconfiguration (§5.6)

**Improving input effectiveness.** Higher entropy of the PRNG leads to lower input effectiveness (CH2), because the probability of finding colliding contract traces decreases. We amend this issue by artificially reducing the PRNG entropy by masking some output bits; lower entropy results in higher input effectiveness but smaller range of tested values. We expect that more sophisticated techniques for creating inputs (e.g., based on symbolic execution) would be able to achieve high effectiveness without manipulating the PRNG.

## 5.3 Executor

The executor has three tasks: (1) collect hardware traces when executing test cases on the CPU (CH3), (2) set the microarchitectural context for the execution (CH4), and (3) eliminate measurement noise (CH5).

**Collecting hardware traces.** To collect traces we employ methods used by side-channel attacks, but in a fully controlled environment. This allows us to record hardware traces corresponding to

the measurements of a powerful worst-case attacker, and spot all consistently-observed leaks via the microarchitectural state. The process involves the following steps:

- (1) Load the test case into a dedicated region of memory,
- (2) Set memory and registers according to the inputs,
- (3) Prepare the side-channel (e.g., prime cache lines),
- (4) Invoke the test case,
- (5) Measure the microarchitectural changes (e.g., probe cache lines) via the side-channel, thus producing a trace.

The measurement (steps 2–5) repeats for all inputs, thus producing a hardware trace for each test case-input pair.

Our implementation supports several measurement modes:

- *Prime +Probe* [33], *Flush +Reload* [52], and *Evict +Reload* [16] modes use the corresponding attack on L1D cache.
- In *\*+Assist* mode, the executor includes microcode assists. It clears the “Accessed” bit in one of the accessible pages such that the first store or load triggers an assist<sup>2</sup>.

*Example:* The hardware trace corresponding to running executor in L1D Prime+Probe mode is a sequence of bits, each representing whether a specific cache set was accessed by the test case or not. E.g., the following trace indicates observed memory accesses to sets 0,4,5: 10001100000000000000000000000000

**Setting the microarchitectural context.** We cannot directly control the microarchitectural context before the test execution (CH4). To deal with this, we develop a technique called **priming**, where we collect traces for a large number of pseudorandom inputs (§5.2) to the same test case *in a sequence*. In this way, execution with one input effectively sets the microarchitectural context for the next input. This enables collection of hardware traces with predictors primed in diverse but deterministic fashion, which is key to obtaining traces that are stable enough for equality checks.

Yet priming may result in undesirable artifacts. For this, recall that MRT searches for inputs  $Data_1$  and  $Data_2$  from the same input class, but with divergent hardware traces:

$$Attack(Prog, Data_1, Ctx) \neq Attack(Prog, Data_2, Ctx)$$

Due to priming, however, the contexts for each input is different, and the actual equality check is:

$$Attack(Prog, Data_1, Ctx_1) \neq Attack(Prog, Data_2, Ctx_2)$$

Therefore, the divergence of traces could be caused by differences in the microarchitectural contexts  $Ctx_1$  and  $Ctx_2$ . For example, earlier inputs can train branch predictors in a way that would prevent speculation for the latter inputs.

To filter such cases and verify that the divergence is caused by inputs and by contexts, we swap  $Data_1$  and  $Data_2$  in the priming sequence, which enables us to test  $Data_1$  with the context  $Ctx_2$  and vice versa. That is, we test the following:

$$Attack(Prog, Data_1, Ctx_2) = Attack(Prog, Data_2, Ctx_2) \\ \wedge Attack(Prog, Data_1, Ctx_1) = Attack(Prog, Data_2, Ctx_1)$$

If this condition holds, we discard the divergence as a measurement artifact, otherwise we report a contract violation.

<sup>2</sup>Microcode assist is a situation when the CPU redirects the control to an internal microcode routine to execute a complex operation, such as setting a page table bit.

*Example:* Consider two inputs with the same contract trace but different hardware traces; in the original sequence of inputs, the first was at position 100 ( $i_{100}$ ) and the second at 200 ( $i_{200}$ ). For priming, the executor tests sequences  $(i_1 \dots i_{99}, i_{200}, i_{101} \dots i_{199}, i_{200})$  and  $(i_1 \dots i_{99}, i_{100}, i_{101} \dots i_{199}, i_{100})$ . The executor will consider it a false positive if  $i_{100}$  at position 200 produces the same trace as  $i_{200}$  at position 200, and vice versa.

**Eliminating measurement noise.** Hardware traces in the same input class may also diverge (and thus incorrectly considered as contract violation) due to several additional sources of inconsistencies which we eliminate as follows:

- (1) *Eliminating measurement noise* (CH5). The executor uses performance counters for cache attacks by reading the L1D miss counter before and after probing a cache line. It proved to give more stable results than timing readings.
- (2) *Eliminating external software noise* (CH5). We run the executor as a kernel module (based on nanoBench [2]). A test is executed on a single core, with hyperthreading, prefetching, and interrupts disabled. The executor also monitors System Management Interrupts (SMI) [21] to discard those measurements polluted by an SMI.
- (3) *Reducing nondeterminism* (CH4). We repeat each measurement (50 times in our experiments) after several rounds of warm-up, and discard one-off traces as likely caused by noise. We then take the union of all traces collected from the executions of a test case with *the same input*, which encompasses all consistently observed variants of speculative behavior under different microarchitectural contexts.

*Example:* Consider again the test case in Figure 3. If the branch in line 6 is speculated differently across the runs, one input may produce different traces:

```
0000101000000100000000000000000001
```

```
0000100000000100000000000000000001
```

The first trace is with a misprediction (cache set 7), and the second without. The merged trace is:

```
0000101000000100000000000000000001
```

Discarding all outliers observed only once during the test might miss rare cases that reveal real leaks. However, we found it necessary from the practical perspective: each reported violation requires manual investigation. Since the outliers turned out to be notoriously hard to reproduce and verify, we opted to focus on the leaks that are easier to distinguish from the noise.

## 5.4 Model

Model’s task is to automate the collection of contract traces (CH6). We achieve this by executing test cases on an ISA-level emulator modified according to the contract. The emulator implements the contract’s execution clause, such as exploring all speculative execution paths, followed by a rollback, and it collects observations based on the observation clause. The resulting trace is a list of observations collected while executing a test case with a single input. We base our implementation on Unicorn [34], a customizable x86 emulator, modified to implement the clauses listed in §2.3.

**Observation Clauses.** When the emulator executes an instruction listed in the observation clause, it records its exposed information

into the trace. This happens during both normal and speculative execution, unless the contract states otherwise.

*Example:* Consider the test case Figure 3 and the contract *MEM-SEQ*. As prescribed by the contract, the model records the accessed addresses when executing lines 3, 7, 13 (Figure 3). Suppose, the branch (line 4) was not taken; the store (line 3) accessed  $0x100$ ; and the load (line 13) accessed  $0x340$ . Then, the contract trace is  $ctrace=[0x100, 0x340]$ .

**Execution Clauses** are implemented similarly to the speculation exposure mechanism introduced in SpecFuzz [32]: Upon encountering an instruction with a non-empty execution clause (e.g., a branch in *MEM-COND*), the emulator takes a checkpoint. The emulator then simulates speculation as described by the clause until (1) the test case ends, (2) the first serializing instruction is encountered, or (3) the maximum possible speculation depth is reached<sup>3</sup>. Then, it rolls back and continues normal execution.

As multiple mispredictions may happen together, the emulator supports nested speculation through a stack of checkpoints: When starting a simulation, the checkpoint is pushed, and afterwards, the emulator rolls back to the topmost checkpoint.

Practically, however, nested speculations greatly reduce the testing speed, which is why we disable nesting by default. This artificially reduces the amount of permitted leakage by the contract, potentially causing false violations (since hardware traces would still include nested speculations). To identify such false violations, Revizor re-executes all reported violations with nesting enabled.

## 5.5 Analyzer

The analyzer compares traces by using relational analysis (§4). As hardware traces are obtained as the union of observations collected from the same input in different microarchitectural contexts (§5.3), we relax requiring equality of hardware traces to requiring only a subset relation. Specifically, we consider two traces equivalent if every observation included in one trace is also included in the other trace.

The intuition behind the heuristic is as follows. If the mismatch is caused by an inconsistent execution of a speculative path among the inputs, one of the inputs executed fewer instructions, therefore fewer observations would appear in the trace, but those that appear match. In contrast, if the mismatch is caused by a secret-dependent instruction, the traces contain the same number of observations, but their values differ. To validate this intuition, we manually examined multiple such examples and did not observe any real violation.

## 5.6 Test Diversity Analysis

If a testing round did not detect any violation, we need to decide how to improve the chances of finding one in the next round. As we test black-box CPUs we cannot measure coverage of the exercised CPU features to guide the test generation in the next round.

Instead, we seek to estimate the likelihood to exercise new speculative paths with the current configuration of the test case generator by analyzing the diversity of the tests we ran so far (CH1). We capture diversity tests using a new measure called *pattern coverage*,

<sup>3</sup>The speculation depth is a configurable parameter. In our experiments, we used 250 instructions, based on the ROB size in Skylake CPUs.

which counts data and control dependencies that are likely to cause pipeline hazards. We expect higher pattern coverage to correlate with higher chances to surface speculative leaks. Therefore, if a testing round does not improve the pattern coverage of the tests so far, new speculative paths are unlikely to be explored. To facilitate generation of more diverse tests, Revizor then increases the number of instructions and basic blocks per test. We now discuss this approach in more details.

**Patterns of instructions.** We define patterns in terms of instruction *pairs*. To simplify the counting of pattern coverage we require that the instructions are consecutive, which corresponds to the worst case for creating hazards. We distinguish three types:

- (1) A memory dependency pattern is two memory accesses to the same address. We consider 4 patterns: store-after-store, store-after-load, load-after-store, load-after-load.
- (2) A register dependency pattern is when one instruction uses a result of another instruction. We consider 2 patterns: dependency over a general-purpose register, and over FLAGS.
- (3) A control dependency pattern is an instruction that modifies the control flow followed by any other instruction. In this paper we consider 2 patterns: conditional and unconditional jumps. Larger instruction sets may include indirect jumps, calls, returns, etc.

We say that a program with an input *matches* a pattern if that pattern is found in two consecutive instructions in the corresponding instruction stream. Since a single input cannot form a counterexample, a pattern is *covered* if a program and two inputs in the same input class match the pattern.

To provide opportunities for interaction between different speculation types, we count not just individual patterns, but also their combinations.

**Implementation.** We implement tracking of patterns as part of the Model (§5.4): While collecting contract traces of a test case, the model also records the executed instructions and the addresses of memory accesses. These data are later analyzed to find the patterns in the instruction streams.

**Coverage Feedback.** We use pattern combination coverage as feedback to the test generator. We begin with test cases of size  $n$  and with at most  $m$  basic blocks, tested with  $k$  inputs (e.g., 10 instructions, 2 blocks, 50 inputs per test case). We continue until all individual patterns are covered. Then, we increase the sizes by constant factors (e.g., 15 instructions, 3 blocks, 75 inputs), and continue testing until all combinations of 2 patterns are covered, and so on.

## 5.7 Postprocessor

When a violation is detected, the test case is passed to the postprocessor, which minimizes the test case in three stages:

First, the postprocessor creates a minimal input sequence: It removes inputs until it finds the smallest sequence to correctly prime the microarchitectural state for the violation. Second, it creates a minimal test case: It removes one instruction at a time while checking for violations. Third, it minimizes the speculative part: It adds LFENCES, starting from the last instruction, while checking for

```

1 AND RAX, 0b111111000000 ; LFENCE
2 LOCK SUB byte ptr [R14 + RAX], 35
3 JNS .bb1 ; LFENCE
4 JMP .bb2
5 .bb1: AND RCX, 0b111111000000
6 REX SUB byte ptr [R14 + RCX], AL
7 .bb2: LFENCE

```

**Figure 4: Minimized test case, representative of Spectre V1.**

violations. The resulting region without fences is the location of leakage.

*Example:* Figure 4 is a minimized version of Figure 3. The highlighted region without LFENCES is the location of leakage: The store (line 2) delays the jump (line 3), thus sufficiently prolonging the speculation. The jump mispredicts and goes to line 5. This causes a speculative execution of SUB (line 6), which has a memory operand and thus leaks the value of RCX.

## 6 EVALUATION

In this section, we demonstrate Revizor’s ability to expose contract violations and automatically identify speculative execution vulnerabilities in two generations of Intel CPUs.

### 6.1 Experimental Setup

We test multiple CPUs, ISA subsets, and threat models against several contracts. The experiments are summarized in Table 2.

**CPUs (rows 1 and 2).** We run our experiments on two machines. The first has Intel Core i7-6700 CPU (Skylake), the second an Intel Core i7-9700 CPU (Coffee Lake). We analyze Skylake with Spectre V4 microcode patch enabled and disabled. Coffee Lake has a hardware MDS patch.

**Instruction Sets (row 3).** We build our test cases from the following subsets of x86<sup>4</sup>:

- AR: in-register arithmetic, including logic and bitwise;
- MEM: memory operands and loads/stores;
- VAR: variable-latency operations (divisions).
- CB: conditional branches;

This totals in the following number of unique instructions: AR—325; AR+MEM—678; AR+MEM+VAR—687; AR+CB—359; AR+MEM+CB—710, AR+MEM+CB+VAR—719.

We select these particular subsets of instructions to structure the description of results. As we will see next, each of them surfaces a different type of contract violations.

**Threat Models (row 4).** We tested contracts against two threat models, *Prime+Probe* and *Prime+Probe+Assist* (see §5.3). Note that Flush/Evict+Reload would produce equivalent traces, as we use a 4KB sandbox, and the 64 L1D cache sets (observed by P+P) correspond to 64 memory blocks in a 4KB region (observed by F+R).

**Configuration.** Generation started from 8 instructions, 2 memory accesses, and 2 basic blocks per test case; 2 bits of input entropy; 50 inputs per test case. The parameters increased over testing rounds.

<sup>4</sup>We do not consider bit count, bit test, and shift instructions because Unicorn sometimes emulates them incorrectly.

**Table 2: Description of the experimental setups.**

|                 | Target 1    | Target 2 | Target 3   | Target 4   | Target 5  | Target 6      | Target 7           | Target 8    |
|-----------------|-------------|----------|------------|------------|-----------|---------------|--------------------|-------------|
| CPU             | Skylake     |          |            |            |           |               | Skylake            | Coffee Lake |
| V4 patch        | off         |          |            | on         |           |               | on                 |             |
| Instruction Set | AR          | AR+MEM   | AR+MEM+VAR | AR+MEM+VAR | AR+MEM+CB | AR+MEM+CB+VAR | AR+MEM             |             |
| Executor Mode   | Prime+Probe |          |            |            |           |               | Prime+Probe+Assist |             |

**Table 3: Testing results.** ✓ means Revizor detected a violation; × means Revizor detected no violations within 24h of testing. In parenthesis are Spectre-type vulnerabilities revealed by the detected violations.

|                     | Target 1 | Target 2 | Target 3                  | Target 4 | Target 5 | Target 6                  | Target 7 | Target 8     |
|---------------------|----------|----------|---------------------------|----------|----------|---------------------------|----------|--------------|
| <i>CT-SEQ</i>       | ×        | ✓ (V4)   | ✓ (V4)                    | ×        | ✓ (V1)   | ✓ (V1)                    | ✓ (MDS)  | ✓ (LVI-Null) |
| <i>CT-BPAS</i>      | ×        | ×        | ✓ (V4-var <sup>**</sup> ) | ×        | ✓ (V1)   | ✓ (V1)                    | ✓ (MDS)  | ✓ (LVI-Null) |
| <i>CT-COND</i>      | ×        | ✓ (V4)   | ✓ (V4)                    | ×        | ×        | ✓ (V1-var <sup>**</sup> ) | ✓ (MDS)  | ✓ (LVI-Null) |
| <i>CT-COND-BPAS</i> | ×        | ×        | ✓ (V4-var <sup>**</sup> ) | ×        | ×        | ✓ (V1-var <sup>**</sup> ) | ✓ (MDS)  | ✓ (LVI-Null) |

\* we did not repeat the experiment as a stronger contract was already satisfied.

<sup>\*\*</sup> the violation represents a novel speculative vulnerability.

## 6.2 Testing Results

We report our findings when testing the targets in Table 2 against different contracts. We tested each for 24 hours or until the first violation was found. The results are in Table 3.

**Target 1: Baseline.** As a baseline, we test the most narrow instruction set AR containing only arithmetic operations on Skylake (with V4 patch disabled) using the weakest threat model (P+P without assists). We expect the target to comply with the most restrictive contract (*CT-SEQ*). The experiments confirm it: Revizor did not detect violations (column 1 of Table 3). Since other contracts are more liberal, the target also complies with more liberal contracts. This experiment shows that Revizor successfully mitigates measurement noise and filters the artifacts of non-deterministic execution, producing no false violations.

**Target 2: Memory Accesses.** When augmenting the instruction set with memory accesses to AR+MEM (for the same CPU and threat model), Revizor detects violations of *CT-SEQ* and *CT-COND*. Upon manual inspection, we identify those violations as representative of Spectre V4 (Speculative Store Bypass) [14]. Revizor does not detect violations of *CT-BPAS* and *CT-COND-BPAS*, which is expected as they permit the store bypass<sup>5</sup>.

**Target 3: Variable-latency Instructions.** When further augmenting the instruction set with divisions (the only variable-latency instructions in the base x86 [1]) to AR+MEM+VAR, Revizor finds violations of *CT-BPAS* and *CT-COND-BPAS*. Upon inspection, they reveal a *novel variant of Spectre V4* that leaks the timing of division (*not* permitted to be exposed according to the contract). We discuss this variant in §6.3.

**Target 4: V4 Patch.** We change the experiment described in Target 3 by enabling the V4 patch on Skylake. Our experiments do not surface any contract violations, showing that the V4 patch is effective, also against the novel V4 variant.

**Targets 5–6: Conditional Branches.** When augmenting AR+MEM with conditional branches to AR+MEM+CB, Revizor detects violations of *CT-SEQ* and *CT-BPAS*. Upon inspection, these are representative of Spectre V1 [23]. Revizor detects no violations of *CT-COND* and *CT-COND-BPAS*, which is expected as the contracts permit exposing accesses during the execution of a mispredicted branch.

When further augmenting the instruction set with variable-latency instructions to AR+MEM+CB+VAR, Revizor detects violations of *CT-COND* and *CT-COND-BPAS*. Similar to Target 3, the violations represent novel variants of Spectre V1.

**Target 7: Microcode Assists.** We now perform experiments with a different threat model, corresponding to an adversary that can cause microcode assists. To test the assists in isolation, we test AR+MEM, and we enable V4 patch to avoid violations caused by V4. Revizor now detects violations of all contracts, which we identify as representative of MDS [7, 40].

**Target 8: MDS Patch.** We repeat the experiment in Target 7, but now on Coffee Lake, which has a hardware MDS patch. Revizor detected violations on it as well, which we identify as LVI-Null [43], a known vulnerability of the MDS patch.

**Summary.** We see that Revizor successfully discovered several known and also unknown vulnerabilities, fully automatically, without manual intervention.

## 6.3 Novel Variants Discovered

Revizor discovers two new types of speculative leakage of the instruction latency. As they represent variations on Spectre V1 and V4, and the existing defences prevent them, we did not report them to Intel. Yet they should be considered when developing future defences, hence we describe them next.

The latency of some operations (e.g., division) depends on their operand values. The timing difference exposes the values to the attacker who can measure the program’s execution time. However, as Revizor discovered, the timing can also impact the cache state, thus leaking through caches as well.

Figure 5 shows a simplified version of the V1 variant. The key observation is that leakage happens due to a race condition:

<sup>5</sup>During the Artifact Evaluation process, Revizor discovered an unexpected counterexample in this experiment, where Target 2 violates *CT-BPAS*. It is described in §A.6

```

1 b = variable_latency(a)
2 if (...) # misprediction
3   c = array[b] # executed if the latency is short

```

**Figure 5: New Spectre V1 variant (V1-Var), found by Revizor.****Table 4: Detection time: the testing time elapsed before the first detected violation. The numbers are mean over 10 measurements; in parentheses are coefficients of variation. Most vulnerabilities are automatically detected within minutes. The second and third rows show that the detection is fast even with multiple leakage types in a test case.**

| Contract-permitted leakage | Detection time     |                    |                     |                     |
|----------------------------|--------------------|--------------------|---------------------|---------------------|
|                            | V4-type (Target 2) | V1-type (Target 5) | MDS-type (Target 7) | LVI-type (Target 8) |
| None                       | 73m 25s (.7)       | 4m 51s (.9)        | 5m 35s (.7)         | 7m 40s (1.1)        |
| V4                         | N/A                | 3m 48s (.7)        | 6m 37s (.8)         | 3m 06s (1.0)        |
| V1                         | 140m 42s (.6)      | N/A                | 7m 03s (.8)         | 3m 22s (.3)         |

- if the variable-latency operation (line 1) is faster than the branch instruction (line 2), then the memory access (line 3) could leave a speculative cache trace.
- otherwise, the speculation will be squashed before the operation completes, and the memory access will not be executed.

As such, the hardware traces expose not only the accessed address, but also the latency of the operation at line 1.

The discovered V4 variant exploits the same race condition; we expect it to affect other speculative vulnerabilities as well.

## 6.4 Validating Assumptions about Speculative Execution

Several defence proposals (STT [53], KLEESpectre [46]) assume that stores do not modify the cache state until they retire. We use Revizor to validate this assumption. We modify *CT-COND* to capture this assumption in the contract trace, and test our CPUs against it. Revizor discovers no violations in Skylake, but finds a counterexample on Coffee Lake. It looks similar to Spectre V1, except the trace is left by a speculative *store*. This is an evidence that the assumption is wrong and speculative stores *can* modify the cache state. Notably, this result has been predicted by previous work, CheckMate [41].

## 6.5 Detection Time

We next measure the time required to find a counterexample. We test each of the targets in §6.2 that had violations (Targets 2, 5, 7, 8) <sup>6</sup> against *CT-SEQ* for 10 times and report the average time until the first violation (row 1 of Table 4).

Revizor detected most violations in under 10 minutes while still using short test cases. This demonstrates the importance of diversity-driven feedback. Revizor took longer to find V4-like violations as they require a longer speculation window, and the hardware predictor is less prone to misprediction.

<sup>6</sup>We did not measure the detection time of the variants discovered in Targets 3 and 6 as they are too rare for repeated measurements.

**Table 5: Detection of known vulnerabilities on manually-written test cases. # Inputs is the average minimal number of random inputs necessary to surface a violation.**

| Violation Type | V1 [23] | V1.1 [22] | V2 [23] | V4 [14] | V5-ret [24, 27] | MDS-LFB [40, 44] | MDS-SB [7] |
|----------------|---------|-----------|---------|---------|-----------------|------------------|------------|
| # Inputs       | 6       | 6         | 4       | 62      | 2               | 2                | 12         |

```

1 a = array1[b]
2 if (...)
3   c = array2[a]

```

**(a) CT-SEQ violation**

```

1 if (...)
2   a = array1[b]
3   c = array2[a]

```

**(b) ARCH-SEQ violation****Figure 6: Subtle difference in sensitivity of contracts.**

**Coping with multiple types of speculation leakages.** We measure how fast Revizor detects a violation when two types of speculative leakage are present in the test case, but one of them is permitted by the contract; that is, Revizor has to detect an unexpected leakage while ignoring an expected leakage. The second and the third row shows the detection time when Spectre V1 and V4 respectively are permitted by the contract and are present in the test case, while testing against *CT-BPAS* (V4 patch disabled). We observe that these additional leakages did not hinder detection of the vulnerabilities, albeit sometimes slowing down the detection because the model has to execute speculative paths, and having more observations reduces input effectiveness.

**Number of Inputs to Violation.** We analyze the number of random inputs that are required to surface a violation of *CT-SEQ* with manually-written test cases representing Spectre and MDS vulnerabilities. Table 5 reports an average of 100 experiments, each with a different input generation seed. Revizor detected all violations with few inputs (i.e., less than a second), illustrating the importance of further research on targeted test case generation.

## 6.6 Contract Sensitivity

The classic Spectre V1 exploit [23] relies on two speculative loads, where the address of the second leaks the value loaded by the first, as in Figure 6b. Hardware defenses based on speculative taint tracking (STT) [49, 53] prevent such leaks, but they do not intend to prevent leaks of *non*-speculatively loaded data, as in Figure 6a.

*MEM-SEQ* and *CT-SEQ* contracts cannot be used to test STT-like defenses as they forbid speculative leakage of any information (i.e., both examples would violate them). Instead, we implement *ARCH-SEQ* (§2.3), which *permits* exposure of non-speculative data, but *forbids* leakage of speculatively loaded data. When testing Skylake against *ARCH-SEQ*, Revizor indeed reports violations corresponding to the classic V1 gadget (Figure 6b) and does not report violations in Figure 6a.

## 7 SCOPE AND LIMITATIONS

**False contract conformance (false negatives).** In several tests, Revizor did not detect violations (Table 3). This *does not prove* the

absence of leaks: it merely shows that the explored space contained no counterexamples.

We see two potential sources of false negatives when we expand to a broader range of testing targets: (1) Noisy measurements: to detect Meltdown/Foreshadow, the executor will have to handle faults, which may pollute the microarchitectural state and make the hardware traces too noisy. (2) Low frequency of counterexamples: some vulnerabilities require a complicated combination of events to observe the leakage (e.g., CrossTalk [36]) or to trigger speculation (e.g., Floating Point Value Injection [35]). Random sampling may be too slow to find a counterexample that would surface them.

A false negative is also possible when the leak is observable only through a certain side channel (other than L1D, used in this paper), and this channel is too noisy to produce stable hardware traces. For example, it may be the case for port contention channels. We note, however, that all known speculative vulnerabilities can be observed through multiple channels, hence L1D measurements are sufficient to detect them. Yet it is not guaranteed for the future, currently-unknown vulnerabilities.

**False contract violations (false positives).** If the model incorrectly emulates ISA, it leads to false positives. Due to this reason, we excluded from the tests some instructions that are not implemented correctly in Unicorn. Non-determinism in the executor may also cause false positives. However, we inspected a few counterexamples in each of the experiments described in §6 and found no false positives.

**Generation of effective inputs.** Revizor applies several restrictions to improve input effectiveness (CH2). This limits the test diversity, and might cause false negatives. To eliminate them, future work may develop a targeted generation method that ensures effectiveness via program analysis, similar to Spectector [17] or Scam-V [30].

**Pattern coverage.** We used hazards as a proxy for speculation. Yet a hazard is not a sufficient condition for speculative leakage. For example, to trigger Spectre V1, branch predictor must be mistrained, and the speculation must be long enough to leave a trace. These preconditions are hard to control on commercial CPUs and, thus, high pattern coverage does not guarantee that speculation was exercised. Improved heuristics to estimate the speculation opportunities in generated test cases might lead to better results.

**Other side-channels.** Revizor currently supports only attacks on L1D caches. For other side-channels, we have to implement them within the executor (e.g., execution port attacks require reading of the port load). For certain speculative attacks, the executor would have to be modified (e.g., Meltdown requires handling of page faults).

**Scalability issues.** In future, adding more features to Revizor and expanding the range of testing targets may exacerbate the search complexity. However, some optimizations may balance out the added overheads: While covering more side channels will require more testing time, improving the input generation process will speed up the testing. Moreover, tests in different adversarial scenarios can easily run in parallel, on different machines with the same CPU model.

**Granularity of measurements.** Revizor currently collects hardware traces once, after the execution of a test case. It means that Revizor does not record the information that could be potentially exposed by the order of memory accesses. To observe the order, we could have probed caches concurrently with the test case execution, but it would introduce additional noise. Hence, we opted for probing the final cache state, which is more deterministic, but records less information.

## 8 RELATED WORK

**Black-box detection of microarchitectural leaks.** Several tools test black-box CPUs to find speculative vulnerabilities: Medusa [28] is a fuzzer for detecting variants of MDS. SpeechMiner [51] is a tool to analyze speculative vulnerabilities. Both of them target specific attacks, while Revizor detects vulnerabilities as violations of a contract.

ABSynthe [15] and Osiris [48] automatically discover unknown side channels. In contrast, Revizor detects unknown speculative leaks into a known side-channel (e.g. L1D cache).

Scam-V [30] is a tool for testing CPUs against a model of side-channel leakage. Their approach is similar to MRT, but their leakage model does not encompass speculation and they focus on analyzing simple, in-order CPUs (Cortex-A53) in which they identify unexpected leaks [31].

**White-box detection of microarchitectural leaks.** A number of approaches use white-box information to detect microarchitectural leaks. Fadiheh et al. [12] proposed a SAT-based bounded model checker to find covert channels in RTL designs (in our terminology, they check RTL against *ARCH-SEQ*). CheckMate [41] searches for pre-defined vulnerability patterns in CPU designs. These tools are not applicable to testing of commercial black-box CPUs.

**Detection of architectural vulnerabilities.** Several tools fuzz for architectural vulnerabilities and ISA violations: TestRIG [50] performs random testing of RISC-V designs. Coppelia [55] generates software exploits for CPU designs. RFuzz [25] is a tool for fuzzing on the RTL level.

Formal models of the ISA [4, 11, 13] could be augmented to capture speculation contracts, along the lines of our instrumentation of Unicorn.

**Information-flow checking.** While information-flow checking (verification and testing) of individual programs is a well-established field (see, e.g., [5, 6, 39]) information-flow checking of language runtimes or processors (which requires reasoning about *all* programs) has not been widely studied. Notable approaches are [20], which generate random programs to surface non-interference violations in an information-flow monitor, and [54] who propose to add information flow annotations to Verilog to detect timing leaks at compile time.

**Speculative leaks in software.** Several tools target detection of speculative leaks in software [8, 17, 19, 32, 45, 47]. They all rely on (sometimes implicit) assumptions about the speculation in hardware, see [9] for an overview. Revizor gives a first principled foundation for validating such assumption on black-box CPUs.

## 9 CONCLUSION

We presented Model-based Relational Testing (MRT), a technique to detect violations of speculation contracts in black-box CPUs. We implemented MRT in a framework called Revizor, and used it to test Intel CPUs against a wide range of contracts.

Our experiments show that Revizor effectively finds contract violations without reporting false positives. The detected violations include known vulnerabilities such as Spectre, MDS, and LVI, as well as novel variants. This demonstrates that MRT is a promising approach for third-party assessment of microarchitectural security in black-box CPUs.

Our work opens several avenues for future research, such as white-box analysis of emerging CPUs and mechanisms for secure speculation, coverage, and targeted testing, for which the open-source release of Revizor will provide a solid foundation.

## ACKNOWLEDGMENTS

We would like to thank Caroline Trippel and the anonymous reviewers for the constructive feedback, and Amaury Chamayou, Sylvan Clebsch, Manuel Costa, Cédric Fournet, Marco Guarnieri, Nuno Lopes, Saidgani Musaev, Robert Norton-Wright, and Alex Shamis for discussions and encouragement.

This work was funded in part by DFG grant 389792660 as part of TRR 248 (CPEC); the Cluster of Excellence EXC 2050/1 (CeTI, project ID 390696704, as part of Germany's Excellence Strategy); the Cloud-KRITIS Project funded by the Sächsische Aufbaubank. This work was also supported by the Technion Hiroshi Fujiwara Cyber Security Research Center and the Israel National Cyber Directorate. We gratefully acknowledge support from Israel Science Foundation (Grant 1027/18).

## A ARTIFACT APPENDIX

### A.1 Abstract

The artifact for this paper includes the source code of Revizor, a set of scripts for reproducing the results, and a description of how to use them. They help to reproduce the contract violations described in the paper and validate the claimed fuzzing speed. §A.6 additionally describes a new violation discovered during the artifact evaluation.

### A.2 Artifact Meta-Information

- **Algorithm:** Random testing of CPUs
- **Hardware:** x86 Intel CPU
- **Metrics:** Detected contract violations and testing speed
- **Output:** The test results and the violating test cases
- **How much disk space required?:** less than 1GB
- **How much time is needed to prepare workflow (approximately?):** 1 hour
- **How much time is needed to complete experiments (approximately?):** 10 days
- **Publicly available?:** Yes
- **Code licenses (if publicly available?):** MIT
- **Workflow framework used?:** Yes
- **Archived (provide DOI?):** 10.5281/zenodo.5865606

### A.3 Description

Below is a brief description of the artifact. You can find more details in the artifact's README file.

#### How to access:

[github.com/hw-sw-contracts/revizor-artifact](https://github.com/hw-sw-contracts/revizor-artifact)

**Hardware dependencies.** The artifact requires at least one physical machine with an Intel CPU and with root access. Preferably, there should be two machines, one with an 8th generation (or earlier) Intel CPU and another with a 9th gen (or later) Intel CPU. To have stable results, the machine(s) should not be actively used by any other software.

#### Software dependencies:

- Linux v5.6+ and Kernel Headers
- Unicorn Engine 1.0.2+ with Python bindings
- Python 3.7+ with packages pyyaml, types-pyyaml, numpy, iced-x86, mypy
- Bash Automated Testing System

**System Configuration (Optional):** For more stable results, disable hyperthreading and boot the kernel on a single core.

### A.4 Installation

- (1) Get submodules:

```
# from the project's root directory
> git submodule update --init --recursive
```

- (2) Copy the ISA description:

```
> cp revizor/src/executor/x86/base.xml
revizor/src/instruction_sets/x86
> cp revizor/src/executor/x86/base.xml
x86.xml
```

- (3) Install the executor:

```
> cd revizor/src/executor/x86
> sudo rmmod x86-executor
> make clean && make
> sudo insmod x86-executor.ko
```

### A.5 Evaluation and Expected Results

The results of all next experiments will be stored in a corresponding subdirectory of results/ with a timestamp. For example, if you run Experiment 1 on 01.01.2022 at 13:00, the result will be stored in: results/experiment\_1/22-01-01-13-00

This directory will contain the experiment logs, detected violations, and aggregated results (when applicable).

**A.5.1 Reproducing fuzzing results.** The following script will test each of the target-contract combinations in Table 3:

```
./experiment_1_main/run.sh
```

Note that the last target (here called target7-8) is dependent on the machine. If you execute the script on an 8th gen (or earlier) CPU, it will correspond to Target 7 in the table. Otherwise, it will correspond to Target 8.

**Note:** The violations of Targets 3 and 6 (called V1-var and V4-var in the paper) are very rare, and there is only a low chance that you will be able to reproduce them. Unfortunately, such unpredictability of the results is an unavoidable consequence of random testing.

**A.5.2 Reproducing speculative store eviction.** For this experiment, you will need a 9th gen Intel CPU or later (in the paper, we tested i7-9700).

To reproduce the violation reported in §6.4, execute the following script, which will test the CPU against a version of CT-COND that does not permit cache eviction by speculative stores.

```
./experiment_2_speculative_store_eviction/run.sh
```

The expected result is that the execution detects a violation within an hour. (If you run this command on an earlier Intel CPU, the expected result is no violations.)

**A.5.3 Fuzzing speed and detection time.**

- (1) To measure the fuzzing speed, simply run Revizor for an hour in a configuration that does not find violations:

```
> ./revizor/src/cli.py fuzz -s x86.xml
-i 200 -n 100000 --timeout 3600
-v -c test-nondetection.yaml
```

Upon completion, Revizor will report the number of executed test cases and the number of inputs.

- (2) To measure the detection speed, execute:

```
./experiment_3a_detection_speed/run.sh
```

The results are expected to approximately match Table 4. The reported numbers are the mean values of the amount of time to detect each of the violations. The meaning of the rows called "mds-\*" depends on the target machine: If the experiment is executed on an 8th gen (or earlier) CPU, they represent MDS-type vulnerabilities. Otherwise, they represent LVI-type.

- (3) To measure the detection speed on handwritten test cases, execute:

```
./experiment_3b_handwritten_test_cases/run.sh
```

The results are expected to approximately match Table 5. The reported numbers are the average, median, minimum, and maximum number of inputs that was required to detect each of the violations with the given test case. The exact numbers will differ slightly with each execution of this experiment, because the input generation seeds are generated randomly. Note: The last two test cases (MDS-SB and MDS-LFB) work only on an 8th gen (or earlier) Intel CPU, because the later generations are patched against MDS.

**A.5.4 Reproducing ARCH-SEQ violations.** To test the CPU against CT-SEQ and ARCH-SEQ, execute:

```
./experiment_4_arch_vs_ct/run.sh
```

The expected result is that both contracts are violated. You can find the counterexamples for both contracts in the results' directory, named ct-seq-violation.asm and arch-seq-violation.asm.

## A.6 Novel Variant of Store Bypass

When an anonymous reviewer evaluated the first experiment, they encountered a violation of CT-BPAS by Target 2, which we did not observe in our previous experiments. Under investigation, it appeared to be a new variant of Speculative Store Bypass. The existing microcode patch provided by Intel mitigates this variant.

Below is a pseudocode of this violation:

```
1. *addr_slow = new_value;
2. x1 = *addr_fast;
3. x2 = *addr_slow;
4. y = array[x1 - x2];
```

Here, `addr_fast` is a pointer to an address in memory initialized with `old_value`. When this code is executed, the pointer is already assigned with the address. `addr_slow` is another pointer assigned with the same address. However the address is calculated dynamically and hence takes more time to resolve.

At line 1, the store overrides the memory value with `new_value`. As the address take a long time to calculate, this store is delayed.

At line 2, the load fetches from the address. The CPU may make a prediction that `addr_fast` and `addr_slow` do not alias, and proceed to speculatively fetch the now-outdated `old_value`; this is the original Speculative Store Bypass.

At line 3, the CPU detects that lines 1 and 3 use the same address, and forwards the `new_value` directly, without waiting for the address to resolve.

As a result, two consecutive loads from the same address speculatively return two different values. The difference between the values is exposed into a side channel by line 4.

Revizor labeled it as a violation of CT-BPAS because the leakage is only possible when one of the loads bypasses the store, but not both of them. This constitutes a violation of CT-BPAS, where *all* loads can bypass an aliasing store.

## REFERENCES

- [1] Andreas Abel and Jan Reineke. 2019. uops.info: Characterizing latency, throughput, and port usage of instructions on Intel microarchitectures. In *ASPLOS*.
- [2] Andreas Abel and Jan Reineke. 2020. nanoBench: A low-overhead tool for running microbenchmarks on x86 systems. In *ISPASS*.
- [3] Jade Alglave. 2012. A formal hierarchy of weak memory models. *Formal Methods in System Design* (2012).
- [4] Alasdair Armstrong, Thomas Bauereiss, Brian Campbell, Alastair Reid, Kathryn E. Gray, Robert M. Norton, Prashanth Mundkur, Mark Wassell, Jon French, Christopher Pulte, Shaked Flur, Ian Stark, Neel Krishnaswami, and Peter Sewell. 2019. ISA Semantics for ARMv8-a, RISC-V, and CHERI-MIPS. In *POPL*.
- [5] Michael Backes, Boris Köpf, and Andrey Rybalchenko. 2009. Automatic discovery and quantification of information leaks. In *2009 30th IEEE Symposium on Security and Privacy*. IEEE, 141–153.
- [6] Gilles Barthe, Pedro R D'argenio, and Tamara Rezk. 2011. Secure information flow by self-composition. *Mathematical Structures in Computer Science* 21, 6 (2011), 1207–1252.
- [7] Claudio Canella, Daniel Genkin, Lukas Giner, Daniel Gruss, Moritz Lipp, Marina Minkin, Daniel Moghimi, Frank Piessens, Michael Schwarz, Berk Sunar, Jo Van Bulck, and Yuval Yarom. 2019. Fallout: Leaking Data on Meltdown-resistant CPUs. In *CCS*.
- [8] Sunjay Cauligi, Craig Disselkoben, Klaus v. Gleissenthall, Deian Stefan, Tamara Rezk, Gilles Barthe, Dean Tullsen, Deian Stefan, Tamara Rezk, and Gilles Barthe. 2020. Constant-Time Foundations for the New Spectre Era. In *PLDI*.
- [9] Sunjay Cauligi, Craig Disselkoben, Daniel Moghimi, Gilles Barthe, and Deian Stefan. 2021. SoK: Practical Foundations for Spectre Defenses. *arXiv:2105.05801* [cs.CR]
- [10] Michael R. Clarkson and Fred B. Schneider. 2010. Hyperproperties. *Journal of Computer Security* (2010).
- [11] Ulan Degenbaev. 2012. *Formal Specification of the x86 Instruction Set Architecture*. Ph.D. Dissertation. Universität des Saarlandes.
- [12] Mohammad Rahmani Fadiheh, Dominik Stoffel, Clark W. Barrett, Subhasish Mitra, and Wolfgang Kunz. 2019. Processor Hardware Security Vulnerabilities and their Detection by Unique Program Execution Checking. In *DATE*.
- [13] Shilpi Goel, Warren A. Hunt, and Matt Kaufmann. 2017. *Engineering a Formal, Executable x86 ISA Simulator for Software Verification*.
- [14] Project Zero Google. 2018. Speculative Execution, Variant 4: Speculative Store Bypass. <https://bugs.chromium.org/p/project-zero/issues/detail?id=1528>. Accessed: May, 2021.
- [15] Ben Gras, Cristiano Giuffrida, Michael Kurth, Herbert Bos, and Kaveh Razavi. 2020. ABSynthe: Automatic Blackbox Side-channel Synthesis on Commodity

- Microarchitectures. In *NDSS*.
- [16] Daniel Gruss, Raphael Spreitzer, and Stefan Mangard. 2015. Cache Template Attacks: Automating Attacks on Inclusive Last-Level Caches. In *Usenix Security*.
  - [17] Marco Guarnieri, Boris Köpf, Jose F. Morales, Jan Reineke, and Andres Sanchez. 2020. SPECTECTOR: Principled Detection of Speculative Information Flows. In *S&P*.
  - [18] Marco Guarnieri, Boris Köpf, Jan Reineke, and Pepe Vila. 2021. Hardware-Software Contracts for Secure Speculation. In *S&P*.
  - [19] Shaobo He, Michael Emmi, and Gabriela Ciocarlie. 2020. ct-fuzz: Fuzzing for Timing Leaks. In *ICST*.
  - [20] Catalin Hritcu, John Hughes, Benjamin C Pierce, Antal Spector-Zabusky, Dimitrios Vytiniotis, Arthur Azevedo de Amorim, and Leonidas Lampropoulos. 2013. Testing noninterference, quickly. *ACM SIGPLAN Notices* 48, 9 (2013), 455–468.
  - [21] Intel Corporation. 2019. *Intel® 64 and IA-32 Architectures Software Developer's Manual*.
  - [22] Vladimir Kiriansky and Carl Waldspurger. 2018. Speculative Buffer Overflows: Attacks and Defenses. *arXiv* (2018). [arXiv:1807.03757](https://arxiv.org/abs/1807.03757)
  - [23] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. 2019. Spectre Attacks: Exploiting Speculative Execution. In *S&P*.
  - [24] Esmaeil Mohammadian Koruyeh, Khaled N Khasawneh, Chengyu Song, and Nael Abu-Ghazaleh. 2018. Spectre Returns! Speculation Attacks using the Return Stack Buffer. In *WOOT*.
  - [25] Kevin Laeuffer, Jack Koenig, Donggyu Kim, Jonathan Bachrach, and Koushik Sen. 2018. RFUZZ: Coverage-directed fuzz testing of RTL on FPGAs. In *ICCAD*.
  - [26] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Anders Fogh, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, and Mike Hamburg. 2018. Meltdown: Reading Kernel Memory from User Space. In *Usenix Security*.
  - [27] Giorgi Maisuradze and Christian Rossow. 2018. ret2spec: Speculative Execution Using Return Stack Buffers. In *CCS*.
  - [28] Daniel Moghimi, Moritz Lipp, Berk Sunar, and Michael Schwarz. 2020. Medusa: Microarchitectural Data Leakage via Automated Attack Synthesis Background Superscalar Memory Architecture. In *Usenix Security*.
  - [29] Alon Naveh, Efraim Rotem, Avi Mendelson, Simcha Gochman, Rajshree Chabukswar, Karthik Krishnan, and Arun Kumar. 2006. Power and Thermal Management in the Intel Core Duo Processor. *Intel Technology Journal* (2006).
  - [30] Hamed Nemati, Pablo Buiras, Andreas Lindner, Roberto Guanciale, and Swen Jacobs. 2020. Validation of Abstract Side-Channel Models for Computer Architectures. In *CAV*.
  - [31] Hamed Nemati, Roberto Guanciale, Pablo Buiras, and Andreas Lindner. 2020. Speculative Leakage in ARM Cortex-A53. *arXiv* (2020). [arXiv:2007.06865](https://arxiv.org/abs/2007.06865)
  - [32] Oleksii Oleksenko, Bohdan Trach, Mark Silberstein, and Christof Fetzer. 2020. SpecFuzz: Bringing Spectre-type vulnerabilities to the surface. In *Usenix Security*.
  - [33] Dag Arne Osvik, Adi Shamir, and Eran Tromer. 2006. Cache Attacks and Countermeasures: The Case of AES. In *CT-RSA*.
  - [34] Nguyen Anh Quynh and Dang Hoang Vu. 2015. Unicorn: Next generation CPU emulator framework. In *BlackHat USA*.
  - [35] Hany Ragab, Enrico Barberis, Herbert Bos, and Cristiano Giuffrida. 2021. Rage against the machine clear: A systematic analysis of machine clears and their implications for transient execution attacks. In *30th USENIX Security Symposium (USENIX Security 21)*. 1451–1468.
  - [36] Hany Ragab, Alyssa Milburn, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. 2021. CrossTalk: Speculative Data Leaks Across Cores Are Real. In *S&P*.
  - [37] Jose Rodrigo, Sanchez Vicarte, Pradyumna Shome, Nandeeeka Nayak, Caroline Trippel, Adam Morrison, David Kohlbrenner, and Christopher W Fletcher. 2021. Opening Pandora's Box: A Systematic Study of New Ways Microarchitecture Can Leak Private Data. In *ISCA*.
  - [38] Efraim Rotem, Eliezer Weissmann, Boris Ginzburg, Alon Naveh, Nadav Shulman, and Ronny Ronen. 2019. Mechanism for saving and retrieving micro-architecture context. US Patent App. 16/259,880.
  - [39] Andrei Sabelfeld and Andrew C Myers. 2003. Language-based information-flow security. *IEEE Journal on selected areas in communications* 21, 1 (2003), 5–19.
  - [40] Michael Schwarz, Moritz Lipp, Daniel Moghimi, Jo Van Bulck, Julian Stecklina, Thomas Prescher, and Daniel Gruss. 2019. ZombieLoad : Cross-Privilege-Boundary Data Sampling. In *CCS*.
  - [41] Caroline Trippel, Daniel Lustig, and Margaret Martonosi. 2018. CheckMate: Automated Exploit Program Generation for Hardware Security Verification. In *MICRO*.
  - [42] Eran Tromer, Dag Arne Osvik, and Adi Shamir. 2010. Efficient Cache Attacks on AES, and Countermeasures. *Journal of Cryptology* (2010).
  - [43] Jo Van Bulck, Daniel Moghimi, Michael Schwarz, Moritz Lipp, Marina Minkin, Daniel Genkin, Yuval Yarom, Berk Sunar, Daniel Gruss, Frank Piessens, and Ku Leuven. 2020. LVI: Hijacking Transient Execution through Microarchitectural Load Value Injection. In *S&P*.
  - [44] Stephan van Schaik, Alyssa Milburn, Sebastian Österlund, Pietro Frigo, Giorgi Maisuradze, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. 2019. RIDL: Rogue In-flight Data Load. In *S&P*.
  - [45] Marco Vassena, Klaus V Gleissenthall, Rami Gökhan Kici, Deian Stefan, and Ranjit Jhala. 2020. Automatically Eliminating Speculative Leaks from Cryptographic Code with Blade. *CoRR* (2020).
  - [46] Guanhua Wang, Sudipta Chattopadhyay, Arnab Kumar Biswas, Tulika Mitra, and Abhik Roychoudhury. 2020. KLEESpectre: Detecting information leakage through speculative cache attacks via symbolic execution. *TOSEM* (2020).
  - [47] Guanhua Wang, Sudipta Chattopadhyay, Ivan Gotovchits, Tulika Mitra, and Abhik Roychoudhury. 2019. oo7: Low-overhead Defense against Spectre Attacks. *IEEE Transactions on Software Engineering* (2019).
  - [48] Daniel Weber, Ahmad Ibrahim, Hamed Nemati, Michael Schwarz, and Christian Rossow. 2021. Osiris: Automated Discovery of Microarchitectural Side Channels. In *Usenix Security*.
  - [49] Ofir Weisse, Ian Neal, Kevin Loughlin, Thomas F. Wenisch, and Baris Kasikci. 2019. NDA: Preventing Speculative Execution Attacks at Their Source. In *MICRO*.
  - [50] Jonathan Woodruff, Alexandre Joannou, Peter Rugg, Hongyan Xia, James Clarke, Hesham Almatary, Prashanth Mundkur, Robert Norton-Wright, Brian Campbell, Simon Moore, and Peter Sewell. 2018. TestRIG: Framework for testing RISC-V processors with Random Instruction Generation. <https://github.com/CTSRD-CHERI/TestRIG>. Accessed: May, 2021.
  - [51] Yuan Xiao, Yinqian Zhang, and Radu Teodorescu. 2020. SpeechMiner: A Framework for Investigating and Measuring Speculative Execution Vulnerabilities. In *NDSS*.
  - [52] Yuval Yarom and Katrina Falkner. 2014. Flush+Reload: A High Resolution, Low Noise, L3 Cache Side-channel Attack. In *Usenix Security*.
  - [53] Jiyong Yu, Mengjia Yan, Artem Khyzha, Adam Morrison, Josep Torrellas, and Christopher W. Fletcher. 2019. Speculative Taint Tracking (STT): A Comprehensive Protection for Speculatively Accessed Data. In *MICRO*.
  - [54] Danfeng Zhang, Yao Wang, G. Edward Suh, and Andrew C. Myers. 2015. A hardware design language for timing-sensitive information-flow security. In *ASPLOS*.
  - [55] Rui Zhang, Calvin Deutschbein, Peng Huang, and Cynthia Sturton. 2018. End-to-End Automated Exploit Generation for Validating the Security of Processor Designs. In *MICRO*.